

Diabetic Retinopathy Pipeline

Implementation & Session Documentation

Generated: 2026-09-18 13:50:27

Machine: DESKTOP-39I5E6Q (Windows 11)

Project root: C:\Users\HP\Downloads\dr-project-python-fixed\dr-project-python-master

This document records the work done to take the rebuilt dr/data/ package from an untested checkout to a running, resumable, full-scale training pipeline on this machine - every real bug found along the way, the checkpoint/resume architecture added because this machine cannot stay powered on for the full multi-day run, the CSV export additions, and a live snapshot of where the pipeline stands as of the timestamp above. Re-run this script any time to refresh that snapshot.

1. Environment Setup

The project ships requirements.txt pinned to torch>=2.2 with no Python version constraint; this machine's only available interpreter is Python 3.14, which is new enough that the default PyTorch CUDA wheel index (cu121) has no build for it at all.

- Created .venv on the system Python 3.14 interpreter.
- Installed PyTorch 2.11.0 + torchvision from the cu128 wheel index (download.pytorch.org/whl/cu128) - the cu121/cu124 indices either lack a 3.14 build or lack Blackwell (RTX 50-series) kernel support.
- Installed the remainder of requirements.txt (timm, opencv-python-headless, scikit-learn/-image, pandas, matplotlib, onnx/onnxruntime, kaggle) - all resolved cleanly for 3.14.
- Placed the user-supplied Kaggle API credentials at %USERPROFILE%\kaggle\kaggle.json so scripts/11_download_lesions.py could fetch the real IDRiD and DDR lesion-annotation corpora.

2. GPU Functionality

Verified end-to-end before committing any real compute time: package installation alone does not prove the CUDA kernels actually match this GPU's architecture.

PyTorch build: 2.11.0+cu128

CUDA available: True

Device: NVIDIA GeForce RTX 5060

VRAM: 8.5 GB

Compute capability: sm_120

Verification performed: allocated a tensor on the CUDA device and ran a real matrix multiply on it (not just an availability check) - confirmed the cu128 build's sm_120 kernels execute correctly on this Blackwell-architecture card, which is not guaranteed by an older CUDA build simply importing without error.

Constraint this GPU imposes on the rest of the run: the project's own full-schedule script (run_gpu_full.sh) assumes ≥ 16 GB VRAM at batch-size 32 for a ViT-Large backbone across a 448px global view plus 6 local crops. This card has 8GB, so the training stage uses --batch-size 4 --grad-accum 8 --grad-checkpoint (same effective batch of 32, same total compute, more but smaller steps) as the starting point - see resume_pipeline.ps1.

Observed throughput on this GPU/CPU combination so far:

- Preprocessing (CPU-bound, 24 worker processes): ~ 1.7 - 2.0 images/sec sustained on real 1024px EyePACS images.
- Lesion-expert pretraining (GPU-bound, small model): ~ 8 - 9 images/sec at 448px on the real IDRiD/DDR crop cache.
- Full staged training throughput has not been measured yet - preprocessing must finish first, since training reads from its cache.

3. Real Bugs Found and Fixed

This code had clearly never been run end-to-end against real, full-scale data before - every one of these surfaced only once actual EyePACS/IDRiD/DDR data and a real training step were exercised, not from the project's own synthetic self-test.

3.1 src/dr/data/eyepacs.py

Problem: build_manifest() only searched a few hardcoded subfolders for images and never found this machine's actual layout (resized_train/resized_train/); it silently found 0 of the 35,126 images.

Fix: Added the actual nested layout (and its _cropped variant) to the search list.

3.2 src/dr/data/lesion_datasets.py

Problem: Folder discovery picked the FIRST matching image/mask folder found anywhere in the tree. For IDRiD (three sections, each with its own 'Original Images' folder) this paired images from the wrong section against the mask folder; for DDR (train/test/valid splits) it paired one split's images against another split's masks. Both cases produced 0 (image, mask) pairs.

Fix: Rewrote discovery to merge every matching folder by filename stem instead of picking one arbitrarily. Now correctly finds all 81 IDRiD and 757 DDR annotated images.

3.3 scripts/08_selftest.py

Problem: The crop-centre assertion expected normalised 0-1 coordinates, but generate_lesion_crops() returns pixel coordinates in the cache's own pixel grid (confirmed against how 02_preprocess.py and CachedEyePACS actually consume the values) - a self-test bug, not a production bug.

Fix: Fixed the assertion to check against the cache_size bound instead of [0,1].

3.4 src/dr/modules/a4_backbone_lora.py

Problem: When require_retfound=False and no checkpoint could be resolved at all, load_retfound_weights() returned a partial report dict; the status-string builder unconditionally read fields (matched_tensors, etc.) that only exist on a successful load, crashing with KeyError.

Fix: Branch on whether the load actually succeeded before building the status string; fall back to a clear 'not loaded' message otherwise.

3.5 scripts/10_lesion_pretrain.py

Problem: Called `train_ds.lesion_prevalence()` as a method; it is a `@property` already returning a dict, so calling it raised `TypeError`. The print statement right behind it then zipped a 6-name tuple against the resulting 4-key dict, which would have crashed again on the next attempt. Separately, the training loop read batch keys `'lesion_masks'/'presence'` that the dataset never produces (it yields `'masks'/'valid'`, with no separate presence label at all).

Fix: Removed the erroneous call, fixed the print to iterate the dict directly, and derived presence from the masks (per-channel: any positive pixel in the crop).

3.6 src/dr/modules/external_lesion_batches.py and scripts/05_predict.py

Problem: Both independently treated `generate_lesion_crops()`'s pixel-space centres as 0-1 fractions and multiplied by image height again, pushing every centre out of range. Since both clip bounds are equal in that regime, every crop for a given image collapsed onto the same fixed corner regardless of where the annotated lesion actually was - this fed both the real-mask auxiliary training path and the Objective 4 XAI evidence path.

Fix: Removed the erroneous re-multiplication in both files.

3.7 scripts/03_train.py

Problem: The training loop read a batch key called `'index'` that `CachedEyePACS` never produces (it's called `'cache_row'`) - would have crashed on the first training step.

Fix: Changed the read to `'cache_row'`.

3.8 scripts/03_train.py

Problem: `HardExampleState.empty(len(train_ds.meta))` sized the hard-example-mining arrays to the `TRAIN-SPLIT` count (~28,102), but they are indexed by `cache_row`, a global row index assigned before the train/val/test split (so it ranges over the whole ~35,126-row cache). Any sample whose `cache_row` fell outside the narrower train-count range would index out of bounds.

Fix: Sized the state to `len(train_ds.meta_full)` instead, matching how `CachedEyePACS` already sizes its own internal hardness buffer for the same reason.

4. Checkpoint / Resume Architecture

This machine cannot stay powered on for the multi-hour preprocessing pass or the multi-day training run, so neither script could originally survive an interruption - a shutdown mid-run meant restarting from image 1 or epoch 1. Both now checkpoint and auto-resume.

4.1 scripts/02_preprocess.py

- Writes one row to `preprocess_progress.csv` per successfully processed image (flushed + fsynced every 200 images), and a separate `preprocess_failed.json` for permanently-unreadable/errored images so they are excluded from the dataset without being retried forever on every resume.
- On startup: if the cache files already exist at the exact shape this invocation implies, opens them in `r+` mode (instead of overwriting) and skips every already-completed or already-failed index.
- `meta.csv` (the file training actually reads) is written to a `.tmp` path and atomically renamed into place, so a crash mid-write can never leave a corrupt/partial `meta.csv`.

- Verified live: stopped the real run after ~400 images, restarted the exact same command, and confirmed it printed '400/35126 already cached, continuing with the remainder' instead of reprocessing from scratch.

4.2 scripts/03_train.py

- Saves a full checkpoint.pt after EVERY completed epoch (not only new-best epochs): model weights, optimizer state, LR-scheduler state, EMA shadow weights, the hard-example-mining state, training history, best/best_epoch/stale, and the torch/CUDA/numpy RNG states - written to a .tmp file and atomically renamed, so a shutdown mid-save cannot corrupt the one checkpoint a resume depends on.
- On restart: reconstructs the LoRA adapter architecture deterministically from the saved per-block ranks (bypassing the RNG-sensitive GLA-LoRA calibration pass entirely, since the LoRA layers are real injected modules that model.load_state_dict() needs to already exist), restores every piece of state above, skips any fully-completed stage, and resumes the partially-completed stage at the next epoch with its original optimizer/scheduler state.
- Worst-case loss from an unplanned shutdown: the single epoch in progress, never more.

4.3 scripts/10_lesion_pretrain.py

Given the same per-epoch checkpoint treatment for consistency, though its crop-cache reuse already made a full re-run cheap.

4.4 resume_pipeline.ps1

A single launcher at the project root chaining self-test -> preprocessing -> lesion pretraining -> staged training -> the full evaluation/reporting pipeline. Safe to re-run after any restart - each stage auto-detects and continues from its own checkpoint. Usage:

```
powershell -ExecutionPolicy Bypass -File resume_pipeline.ps1
```

Caveat: once training has written its first checkpoint, the arguments inside this script (particularly --batch-size/--grad-accum) should not change - a resumed run reuses the saved LR-schedule state, which a change would throw off. The final evaluation/report stages are not individually checkpointed (only the two multi-hour stages are); if one of those is interrupted, only that one step needs re-running.

5. CSV Export Additions

A shared helper, src/dr/csv_export.py (save_csv_alongside), writes a best-effort tabular CSV next to whichever JSON a script already produces, wherever the data has one obvious shape: a list of flat dicts becomes one row per entry; a dict of dicts becomes one named row per key; a flat dict becomes a single summary row. Verified against each of these shapes before wiring it into every script below.

- 03_train.py: history.csv (per-epoch curve), gla_lora.csv (per-block LoRA ranks/importance)
- 10_lesion_pretrain.py: history.csv, sources.csv
- 07_compare_models.py: comparison.csv (QWK-sorted leaderboard), results.csv (every field), significance.csv
- 04_evaluate.py: evaluation.csv (long format: group/metric/value), evaluation_confusion_matrix.csv
- 06_deploy.py: distilled_metrics.csv, distilled_variants.csv
- 09_screening.py: screening.csv, screening_deployable.csv

- 12_lesion_visibility.py: lesion_visibility.csv
- 13_linear_probe.py: linear_probe_<tag>.csv
- 14/15/18/20/22_*.py: one CSV per objective evidence-pack JSON
- 16_ordinal_calibration.py: ordinal_calibration_results.csv
- 19_objective3_validation.py: objective3_validation_arms.csv
- 21_xai_evaluation.py: xai_evaluation_per_image.csv, xai_evaluation_metrics.csv
- 23_objectives_summary.py: objectives_consolidated.csv

6. Current Pipeline Status (live at generation time)

6.1 Preprocessing

Images cached so far: 35,126 / 35,126

Last log lines:

```
[preproc] 14901/35126  1.9 img/s (this run)  eta 180.3 min
[preproc] 15401/35126  1.9 img/s (this run)  eta 177.1 min
[preproc] 15901/35126  1.9 img/s (this run)  eta 172.4 min
[preproc] 16401/35126  1.9 img/s (this run)  eta 167.8 min
```

6.2 Lesion-Expert Pretraining

COMPLETE - 8 epochs run, best validation Dice 0.1401. Checkpoint: outputs/lesion_pretrain/lesion_encoder.pt

6.3 Staged Training

IN PROGRESS / RESUMABLE - 1 epoch(s) checkpointed so far.

Last completed epoch: 1 stage=s1_frozen QWK=0.3505 score=0.3003

6.4 Disk / Environment

Free disk space: 693.5 GB

GPU: NVIDIA GeForce RTX 5060 (8.5 GB VRAM)

7. Next Steps

- Let preprocessing finish (full 35,126-image, 1024px cache).
- Start scripts/03_train.py with --lesion-encoder pointed at the completed lesion_encoder.pt (already wired into resume_pipeline.ps1), starting at --batch-size 4 --grad-accum 8 --grad-checkpoint for the 8GB card.
- If training OOMs before its first checkpoint, lower --batch-size further and raise --grad-accum to compensate - safe to do at that point since nothing has been checkpointed yet.
- Once training completes all four stages, run the full evaluation/reporting sequence (already chained in resume_pipeline.ps1) and read outputs/objectives_consolidated.json / .csv first.
- Re-run this script (generate_session_report.py) at any point to refresh this document with current progress.

Appendix: Experiment Values Registry

Source: data\experiment_values.json, generated 2026-09-18T13:49:38. Every value below is read from a real config/log/output file at generation time; anything not yet computable is marked PENDING, and anything this codebase does not implement is marked NOT APPLICABLE, rather than estimated or invented.

1 dataset data integrity

dataset name: EyePACS (Kaggle 'Diabetic Retinopathy Detection', high-res 'eyepacs-hi' export) for training/val/test; APTOS 2019 as an external-only evaluation cohort; IDRiD + DDR for lesion annotation supervision (see section 8)

dataset version: Kaggle competition export as downloaded to data/raw/eyepacs-hi (trainLabels.csv, 35,127 labelled rows; 35,126 image files actually present and used)

total image count: 35126

total patient count: 17563

total eye count: NOT APPLICABLE - not implemented in this codebase as a separate figure (each row IS one eye; patient_id is the shared numeric prefix of '<id>_left'/'<id>_right')

image ids patient ids eye ids: stored per-row in data/cache/meta.csv (image_id, patient_id, cache_row columns) - not reproduced here row-by-row (35,126 rows)

original image paths: data/raw/eyepacs-hi/resized_train/resized_train/<image_id>.jpeg

image dimensions native: as downloaded from Kaggle (varies by source camera); resampled into the cache at cache_size below

cache resolution px: 1024

dr grade counts:

0: 25810

1: 2443

2: 5292

3: 873

4: 708

class percentages:

0: 73.48

1: 6.95

2: 15.07

3: 2.49

4: 2.02

train count: 28102

val count full: 3512

val count used for epoch monitoring: 1500

test count: 3512

train class counts: [20641, 1944, 4250, 707, 560]

validation class counts: PENDING - computed after training/evaluation completes (not logged per-class during training; computable from data/cache/meta.csv split=val rows)

test class counts: PENDING - computed after training/evaluation completes (only read at final evaluation, scripts/04_evaluate.py)

split level: patient-level (both eyes of one patient always share a split - src/dr/data/eyepacs.py:split_by_patient)

eye level split: NOT APPLICABLE - not implemented in this codebase (this project deliberately does NOT split by eye, to prevent leakage - see split_level above)

split random seed: 1337

split csvs: NOT APPLICABLE - not implemented in this codebase (the split is a deterministic function of meta.csv + seed, recomputed each run rather than exported as a standalone CSV; the 'split' column IS a permanent field of data/cache/meta.csv)

train val overlap: 0 (patient-level split verified)

train test overlap: 0 (patient-level split verified)

val test overlap: 0 (patient-level split verified)

duplicate image count: NOT APPLICABLE - not implemented in this codebase (no perceptual/hash dedup pass implemented)

duplicate hash count: NOT APPLICABLE - not implemented in this codebase

leakage check result: PASSED - '[split] verified: no patient appears in two splits' (assertion in scripts/02_preprocess.py)

2 preprocessing quality pipeline

global image size px: 448

high res cache size px: 1024

local crop size native px: 320

local crop fed size px: 224

n local crops: 6

crop jitter: 0.12

quality score mean qstar: 0.612

quality score mean legacy: 0.681

q axes mean:

q quality: 0.789

q domain: 0.667

q lesion: 0.453

q blur: 0.436

q illumination: 0.72

q lesion by grade:

0: 0.4531

1: 0.4065

2: 0.4645

3: 0.4417

4: 0.5585

quality by dataset: NOT APPLICABLE - not implemented in this codebase (single-dataset preprocessing run; no per-dataset quality breakdown computed)

gate decisions:

accept:

count: 18501

pct: 52.67

enhance:

count: 16474

pct: 46.9

gate decision per image: stored per-row in data/cache/meta.csv (quality_decision column)

preprocessing runtime per image: ~12.5 CPU-seconds/image single-threaded (measured this session at 1024px, full A1+A2+lesion-priors pipeline)

total preprocessing runtime: ~4.5 hours wall-clock for all 35,126 images with 28 parallel worker processes (measured)

failed preprocessing images: 0 (see data/cache/preprocess_failed.json)

clahe clip limit: 2.5

clahe grid size: 8

illumination norm sigma: 25.0

enhancement algorithm: clahe

large lesion protection:

enabled: True

kernel: 41

strength: 0.6

alpp gate hidden dim: 64

alpp branches: ['norm', 'contrast', 'vessel', 'lesion']

alpp initial bias prior: [1.2, 0.6, 0.3, 0.3]

alpp fusion gamma: 0.6

alpp weights per image by grade by quality: PENDING - computed after training/evaluation completes (per-image ALPP weights are computed on the fly during training/preprocessing, not persisted per-image)

3 alpp verified constants

downsample before gate net px: 128x128 (verified in src/dr/modules/a2_preprocess.py: F.interpolate(..., size=(128,128)))

gate hidden units: 64

initial bias prior: [1.2, 0.6, 0.3, 0.3]

fusion gamma: 0.6

branches: ['norm', 'contrast', 'vessel', 'lesion']

note: these four values were independently cross-checked against the live config.json AND the a2_preprocess.py source for this exact run and all matched what had been previously documented (128x128 / 64 / [1.2,0.6,0.3,0.3] / 0.6)

4 retfound backbone

checkpoint source: bitfount/RETFound_MAE:pytorch_model.bin

checkpoint fallback configured: YukunZhou/RETFound_mae_natureCFP:RETFound_mae_natureCFP.pth

require retfound: True

load report line: [A4] RETFound weights loaded from C:\Users\HP\.cache\huggingface\hub\models--bitfount--RETFound_MAE\snapshots\0f81b6df4222edacc026f9fab2aa81bc71ebb1cf\pytorch_model.bin (matched 294, missing 0, unexpected 0, backbone coverage 100.0%)

backbone coverage pct: 100.0

architecture: vit_large_patch16_224

input resolution global px: 448

input resolution local crop px: 224

patch size: 16 (vit_large_patch16_224 - patch size encoded in model name)

total model parameters: 319612795

checkpoint hash version: HF snapshot 0f81b6df4222edacc026f9fab2aa81bc71ebb1cf (bitfount/RETFound_MAE)

5 dynamic gla lora

enabled: True

rank min: 2

rank max: 16

lora alpha: 16.0

lora dropout: 0.05

lora targets: ['qkv', 'proj']

importance weights:

lam grad: 0.35

lam lesion: 0.45

lam attn: 0.2

lesion layer boost threshold: 0.85

per block table:

block	importance_S	grad_G	lesion_L	attn_A	rank
0	0.7718	1.0	0.5896	0.7824	13
1	0.4332	0.1236	0.7277	0.3121	8
2	0.4265	0.1795	0.8082	0.0	8
3	0.5797	0.1029	0.8289	0.8536	10

4		0.6459		0.164		0.8925		0.9345		11
5		0.6313		0.0993		0.9481		0.8493		11
6		0.6211		0.0926		0.9073		0.9018		11
7		0.6477		0.0561		0.9512		1.0		11
8		0.5821		0.1103		0.9994		0.469		10
9		0.5974		0.1663		1.0		0.446		10
10		0.6575		0.0946		0.9861		0.903		11
11		0.5723		0.0944		0.9645		0.5261		10
12		0.5831		0.046		0.9674		0.6584		10
13		0.5583		0.0401		0.9524		0.5782		10
14		0.5351		0.0062		0.9519		0.5229		9
15		0.4875		0.003		0.9478		0.2995		9
16		0.614		0.0152		0.945		0.9171		11
17		0.5576		0.0173		0.9233		0.6802		10
18		0.4938		0.0041		0.886		0.4684		9
19		0.5119		0.0		0.8463		0.6553		9
20		0.5361		0.0018		0.8584		0.7459		10
21		0.5872		0.0169		0.8496		0.9951		10
22		0.4704		0.0285		0.8196		0.458		9
23		0.0941		0.009		0.0		0.4548		3

trainable lora parameters: 1431552

backbone total parameters: 319612795

lora parameter pct: 0.4479

n blocks at r max: 0

n blocks at r min: 0

calibration seed: 1337

calibration batch size: --calib-batch (default 4)

documented headline values:

backbone params approx: 319.6M

lora params approx: 1431552

lora pct of backbone: 0.45%

block 0 rank: 13

block 21 rank: 10

block 23 rank: 3

block 23 importance: 0.0941

6 training stages

n stages: 4

stages:

name	epochs	train_backbone	train_lora	unfreeze_frac	head_lr	lora_lr	backbone_lr	train_alpp
s1_frozen	5	False	False	0.0	0.0001	0.0	0.0	False
s2_lora	12	False	True	0.0	0.0001	5e-05	0.0	True
s3_unfreeze	14	True	True	0.33	5e-05	5e-05	5e-06	True
s4_full	14	True	True	1.0	2e-05	2e-05	5e-06	True

batch size: 4

grad accum: 8

effective batch size: 32

weight decay: 0.02

warmup frac: 0.1

grad clip norm: 1.0

scheduler: warmup + cosine decay (LambdaLR, per-stage step budget)

amp: True

amp dtype: bfloat16 (CUDA Ampere+/RTX 5060 supports bf16 natively)

gradient checkpointing: True

ema enabled: True

ema decay: 0.995

seed: 1337

num seeds: 1

ladder lr mult: 10.0

init ladder from prior: True

num workers: 16

patience epochs: 8

select objective: ordinal

select weights:

qwk: 0.5

macro f1: 0.3

minority recall: 0.2

cpu utilization monitoring: NOT APPLICABLE - not implemented in this codebase per user instruction (not tracked going forward)

data loading forward backward optimizer time breakdown: NOT APPLICABLE - not implemented in this codebase (not separately instrumented; only aggregate throughput img/s is measured)

7 class imbalance sampling

sampling power: -0.5

formula: $P_i \propto n_{\text{class}(i)}^{\text{power}} * \min((1+H_i)^{\text{hard_eta}}, \text{hard_clip}) * (1+\text{boundary_bonus}*\text{boundary}_i) * (1+\text{lowconf_bonus}*\text{lowconf}_i)$

hard eta: 0.5

hard eps: 0.001
hard ema decay: 0.7
hard clip: 4.0
boundary bonus: 0.6
lowconf bonus: 0.4
class weight power cb focal: -0.25
class counts train: [20641, 1944, 4250, 707, 560]
self test verified ratio: grade4/grade0 sampling-weight ratio = 5.01 exactly (automated self-test, synthetic counts)
self test verified mining lift: 1.82x boundary-error weight lift (automated self-test)
samples per epoch: 6000 (resume_pipeline.ps1 default)
sampler state checkpointed: no - the sampler/hardness EMA state IS checkpointed (HardExampleState in checkpoint.pt), but the WeightedRandomSampler draw sequence itself is not seeded for exact replay across a resume

8 lesion encoder

idrid images: 81

idrid per channel:

MA: 81
HE: 80
EX H: 81
EX S: 40

ddr images: 757

ddr per channel:

MA: 757
HE: 757
EX H: 757
EX S: 757

total annotated images: 838

nv annotation availability: none (neither IDRiD nor DDR annotate neovascularisation - masked out of every loss)

me annotation availability: none (neither corpus annotates macular oedema)

architecture: LesionMoE (src/dr/modules/a3_lesion_moe.py), 1,824,498 trainable parameters, evidence maps at 112px

pretraining seed: 1337

pretraining epochs: 8

history per epoch:

epoch	train_loss	epoch_sec	dice_MA	dice_HE	dice_EX_H	dice_EX_S	dice_NV	dice_ME
-------	------------	-----------	---------	---------	-----------	-----------	---------	---------

```
dice_mean | _annotated_per_channel

1 | 30.707891882693723 | 95.9 | 0.0 | 0.0 | 0.13920166771877798 | 0.0 | nan | nan |
0.034800416929694496 | [756, 756, 756, 756, 0, 0]

2 | 2.50976720407661 | 80.9 | 0.030419715055833654 | 0.0009455226967658058 | 0.33002379380364505 |
0.0 | nan | nan | 0.09034725788906113 | [756, 756, 756, 756, 0, 0]

3 | 2.240821138303378 | 80.7 | 0.0330642060745867 | 0.022294240654497588 | 0.19218370001696736 |
0.0 | nan | nan | 0.06188553668651291 | [756, 756, 756, 756, 0, 0]

4 | 2.094665002901009 | 82.8 | 0.08115838398283876 | 0.02094127266204457 | 0.31449820811550266 |
0.0 | nan | nan | 0.10414946619009649 | [756, 756, 756, 756, 0, 0]

5 | 2.0033256150363536 | 82.3 | 0.06389193136181089 | 0.055255372729273125 | 0.41991267576580604 |
0.0 | nan | nan | 0.1347649949642225 | [756, 756, 756, 756, 0, 0]

6 | 1.9438250305947293 | 82.4 | 0.09528985507246376 | 0.044079672363032285 | 0.31747131747131746 |
0.0018192844147968466 | nan | nan | 0.1146650323304026 | [756, 756, 756, 756, 0, 0]

7 | 1.904215879598807 | 82.8 | 0.08621626516935338 | 0.04618516755052785 | 0.4210731082148047 | 0.0
| nan | nan | 0.1383686352336715 | [756, 756, 756, 756, 0, 0]

8 | 1.888188412517644 | 82.2 | 0.09419770773638969 | 0.04400411099691675 | 0.42226241443256485 |
0.0 | nan | nan | 0.14011605829146784 | [756, 756, 756, 756, 0, 0]
```

best epoch: 8

best mean val dice: 0.14011605829146784

final epoch per channel dice:

```
dice MA: 0.09419770773638969
dice HE: 0.04400411099691675
dice EX H: 0.42226241443256485
dice EX S: 0.0
dice NV: nan
dice ME: nan
dice mean: 0.14011605829146784
```

transfer into main model: [A3] lesion encoder transferred from outputs\lesion_pretrain\lesion_encoder.pt (124 tensors; missing 0, unexpected 0; pretrained on {'idrid': {'images': 81, 'masks_per_channel': {'MA': 81, 'HE': 80, 'EX_H': 81, 'EX_S': 40}}, 'ddr': {'images': 757, 'masks_per_channel': {'MA': 757, 'HE': 757, 'EX_H': 757, 'EX_S': 757}}}, val Dice 0.1401)

documented headline values:

```
tensors transferred: 124
missing: 0
unexpected: 0
best mean val dice approx: 0.1401
EX H dice approx: 0.42
MA dice approx: 0.09
HE dice approx: 0.04
EX S dice approx: 0.002
```

9 cross attention pathology anatomy a5

note: this codebase's cross-attention module is A5 (pathology<->anatomy fusion, src/dr/modules/a5_fusion_gnn.py) - bidirectional attention between lesion-evidence tokens and named anatomical regions, NOT a separate 'global/local/HR' three-way cross-attention (that describes a different architecture this codebase does not have)

n heads: 8

anatomy regions: ['optic_disc', 'macula', 'sup_arcade', 'inf_arcade', 'peripheral']

gnn layers: 2

knn: 6

fused dim: 384

bidirectional: True

probe dim: 128

self test verified shapes: P->A (2,8,53,5) A->P (2,8,5,53) L_PA=1.7229 (automated self-test, synthetic batch of 2)

global lesion hr specific attention maps: NOT APPLICABLE - not implemented in this codebase

10 ordinal coral

n classes K: 5

n thresholds: 4

threshold lr multiplier: 10.0

init from prior: True

uniform init thresholds: [0.0, -0.9740999937057495, -1.948199987411499, -2.9221999645233154]

prior matched thresholds: [-1.0176000595092773, -1.409500002861023, -3.053100109100342, -3.8954999446868896]

decode rule: coral (rank-counting; --decode argmax also available as an ablation)

pi coral vs standard coral: this run uses prior-initialised CORAL ("PI-CORAL" in the objective-1 write-up) by default (init_ladder_from_prior=true); standard uniform-init CORAL is available via --no-init-ladder-from-prior

reachability bounds per grade: PENDING - computed after training/evaluation completes (computed by scripts/16_ordinal_calibration.py and scripts/15_objective1_report.py's decode-reachability sweep, both post-training)

11 dqk loss

enabled: True

weight w dqk: 0.5

dqk vs direct qwk agreement: PENDING - computed after training/evaluation completes (this is exactly what scripts/15_objective1_report.py's decode-ablation compares, post-training)

12 other loss components

learned loss weighting: True

fixed weights if not learned:

w ordinal: 1.0

w hierarchical: 0.5

w boundary: 0.2

w contrastive: 0.2

w lesion: 0.3

w pa: 0.1

w cbf: 0.3

w dqk: 0.5

focal gamma: 2.0

cb beta: 0.999

boundary gamma: 1.5

contrastive temperature: 0.1

lambda xai schedule: [0.05, 0.05, 0.05, 0.05]

lambda xai override this run: constant 0.05 from stage 1 (--xai-weight 0.05, overriding the default 0-at-stage-1/2 schedule) - deliberate, to avoid the documented lambda_XAI=0 failure mode

emd margin losses: NOT APPLICABLE - not implemented in this codebase (this codebase does not implement separate EMD or margin loss terms - boundary_loss/contrastive_loss serve an analogous role)

13 main training history so far

epochs completed this run:

stage	epoch	train_loss	qwk	f1_macro	min_recall	score	accuracy	ref_auc
s1_frozen	1	83.8989	0.3505	0.292	0.1872	0.3003	0.666	0.726

checkpoints saved this run:

stage	epoch
1	1

last live step:

stage: 1

epoch in stage: 2

step: 225

total steps: 1500

train loss: 58.1437

throughput img s: 1.92

monitoring time series file: data/training_monitor.csv (1 rows so far)

learning rate per step: NOT APPLICABLE - not implemented in this codebase (not logged per-step; per-stage LRs are in section 6 config)

14 final dr grading metrics: PENDING - computed after training/evaluation completes (test-set accuracy/balanced-accuracy/macro-F1/weighted-F1/QWK/MAE, per-grade P/R/F1/support, confusion matrix, calibration NLL/ECE/Brier - all from scripts/04_evaluate.py, which runs once after training)

15 raw prediction outputs: PENDING - computed after training/evaluation completes (per-image predictions/probabilities are written by scripts/05_predict.py per image and by scripts/04_evaluate.py in aggregate - not yet run against a finished checkpoint)

16 explainability xai: PENDING - computed after training/evaluation completes (attribution AUROC/Dice/IoU vs the real 838 IDRiD/DDR masks, counterfactual sensitivity - scripts/21_xai_evaluation.py, post-training)

17 uncertainty: PENDING - computed after training/evaluation completes (MC-dropout epistemic uncertainty, decision-gate accept/reject counts on real predictions - needs a finished checkpoint; the self-test already verifies the MECHANISM works on synthetic data)

18 ablation experiments: NOT APPLICABLE - not implemented in this codebase for the full grid requested (Baseline+PI-CORAL/+DQK/+CB-Focal/+EMD/+margin/etc as SEPARATE trained arms) - this run trains ONE configuration with all components on. scripts/07_compare_models.py DOES provide baseline-vs-proposed comparisons (CNN/ Transformer baselines vs this architecture) and is part of the post-training pipeline, but it is not a component-by-component ablation grid.

19 multi seed statistics: NOT APPLICABLE - not implemented in this codebase (this run uses a single seed, 1337; no multi-seed variance/CI study is implemented or planned in this codebase)

20 cross domain evaluation: PARTIAL - scripts/04_evaluate.py scores the trained model against an APTOS-2019 external cache (data/cache_external) as a domain-shift check; a full IDRiD/DDR/EyePACS three-way grading comparison is NOT APPLICABLE - not implemented in this codebase

21 checkpoints reproducibility

checkpoint file: outputs/retfound_plus_laft_xai/checkpoint.pt (every completed epoch, overwritten in place, atomic write)

best checkpoint file: outputs/retfound_plus_laft_xai/best.pt (only when selection score improves)

contents of checkpoint pt: ['model state_dict', 'optimizer state_dict', 'scheduler state_dict', 'EMA shadow weights', 'hard-example-mining state (hardness/seen/boundary/lowconf arrays)', 'training history', 'best score/epoch/stale counter', 'torch/CUDA/numpy RNG state', 'GLA-LoRA ranks', 'original CLI args']

python rng state saved: no - only torch/CUDA/numpy RNG saved (the training loop does not use Python's own random module directly)

dataloader sampler state saved: no - WeightedRandomSampler's draw sequence is not itself seeded/checkpointed for exact replay; the underlying hardness/mining STATE that produces its weights IS checkpointed

dataset split reproducibility: deterministic given (meta.csv contents, seed 1337) via split_by_patient() - re-running produces the identical split

config json: outputs/retfound_plus_laft_xai/config.json (full config, reproduced above section-by-section)

command line args used: see resume_pipeline.ps1's 'Staged training' step for the exact invocation

18 common compute environment

cpu cores logical: 28

ram gb: 31.7

gpu model: NVIDIA GeForce RTX 5060

gpu vram gb: 8.5

gpu compute capability: 12.0 (sm_120, Blackwell)

cuda version: 12.8

pytorch version: 2.11.0+cu128

torchvision version: 0.26.0+cu128

timm version: 1.0.29

opencv version: 5.0.0

python version: 3.14.7

os: Windows-11-10.0.26200-SP0

git commit: a2dc56776db27a0853d2bbcacb765e0f923283c2

git commit short: a2dc567

power plan: High performance (switched from Balanced this session)